DASE 7506 · SEMESTER 1 · SESSION 6

Advanced Machine Learning Methods

Inference-time scaling and RLHF — buying accuracy with compute at answer time, then learning a reward from human preferences.

Jiayu Chen · 7 October 2026 · Raschka, Build a Reasoning Model (From Scratch), chapters 4–5 · RLHF papers

SESSION 6

Today’s Three Blocks

From spending compute at inference time to training on human preferences.

01

Improving Reasoning with Inference-Time Scaling

Chapter 4. Chain-of-thought prompting, temperature and top-p sampling, and self-consistency voting on MATH-500.

50 MIN

02

Inference-Time Scaling via Self-Refinement

Chapter 5. Rule-based and log-probability scoring; a critique-and-revise loop that keeps a revision only if it scores well.

50 MIN

03

Reinforcement Learning from Human Feedback

Preference data, Bradley–Terry reward models, PPO with a KL penalty, DPO, and the bridge to RL with verifiable rewards.

50 MIN

PART II · OVERVIEW

Where Today Sits in the Course

Session 5 built the harness; today we spend compute, then learn from preferences.

SESSION 5 · DONE

What is reasoning in an LLM?

Chapters 1–3: the Qwen3 0.6B base model, the generation loop and the MATH-500 verifier; the CoT and ReAct papers.

SESSION 6 · TODAY

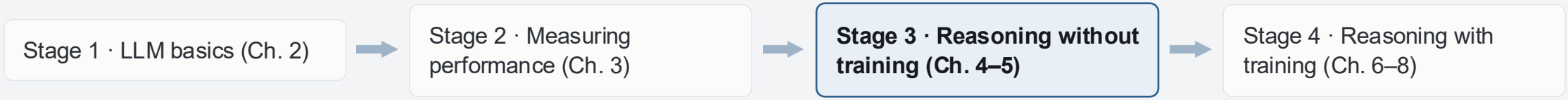
Inference-time scaling and RLHF

Chapters 4–5: sampling, voting, scoring and self-refinement. Then RLHF: reward models, PPO and DPO.

COMING NEXT

Training reasoning models

Chapters 6–8: RL with verifiable rewards (GRPO), improving GRPO, and distilling reasoning traces.



BLOCK I · CHAPTER 4

Improving Reasoning with Inference-Time Scaling

Spend more compute per question — and leave every weight untouched.

CHAPTER 4.1 · FIGURE 4.2

Two Ways to Buy Accuracy

Both spend compute; only one of them changes the weights.

TRAINING-TIME COMPUTE

Spend more before deployment: pretraining, reinforcement learning, distillation (Chapters 6–8).

- Paid once, amortized over every later query
- Changes the weights

INFERENCE-TIME COMPUTE · TODAY

Spend more per question: longer reasoning, more samples, rounds of refinement (Chapters 4–5).

- Paid on every query; grows with traffic
- Weights untouched; also called test-time scaling

Inference-time scaling: more computation at answer time, after training. In practice both are combined: reasoning models are trained heavily, then also think longer.

CHAPTER 4.1 · FIGURE 4.3

Three Methods, Two Patterns

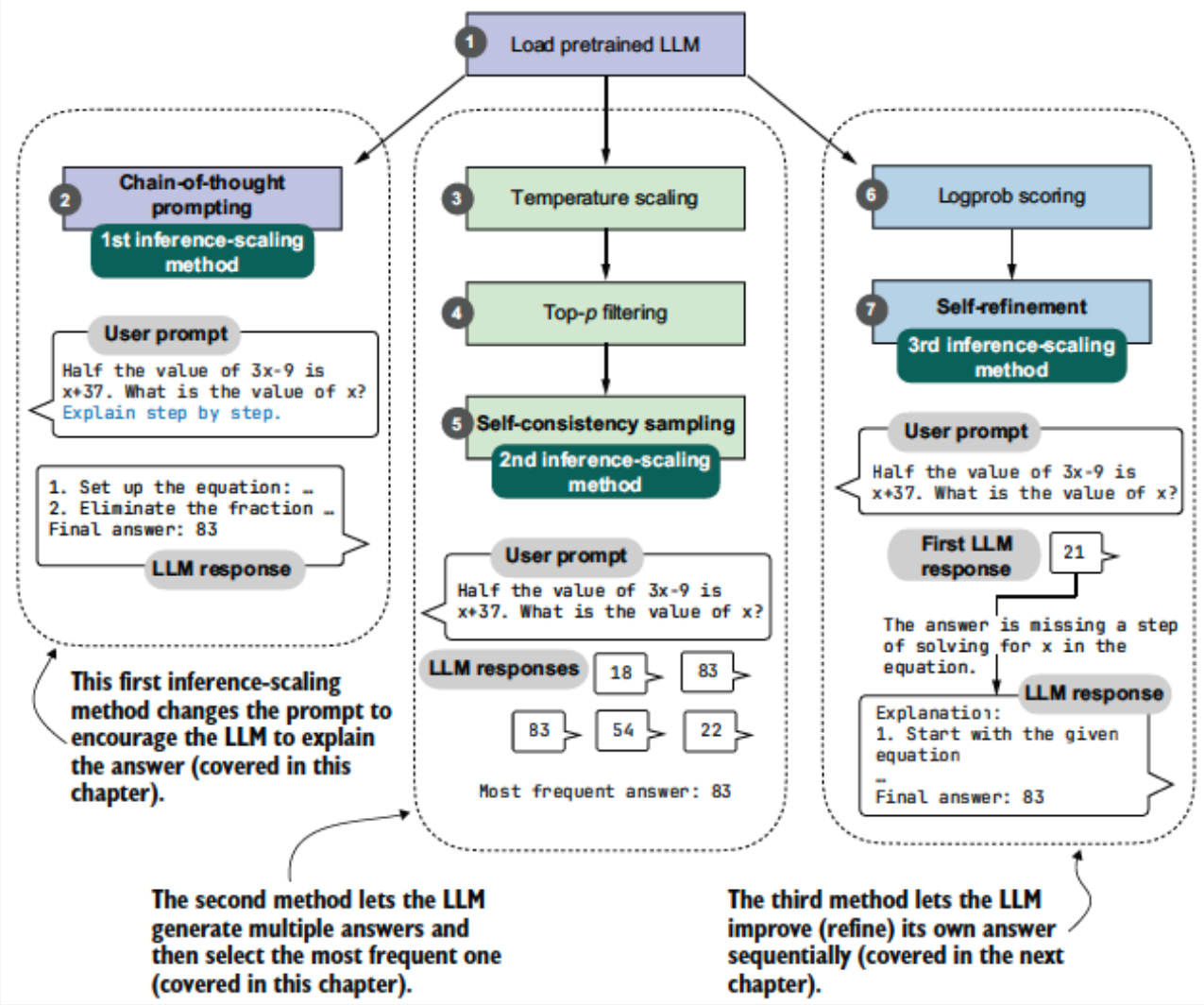
Longer responses or more responses: each buys accuracy with extra tokens.

Method	Pattern	What grows	Where
1 · Chain-of-thought prompting	Sequential: one longer response	Tokens per answer	Chapter 4 · Block I
2 · Self-consistency	Parallel: several responses, then a vote	Number of answers	Chapter 4 · Block I
3 · Self-refinement	Sequential: draft, critique, revision	Rounds of generation	Chapter 5 · Block II

Why these three? Of more than ten techniques the author tested, they improve the 0.6B model, span the main paradigms (longer vs. more responses), and are simple to build from scratch.

CHAPTER 4.1 · FIGURE 4.3

Three Methods, Two Patterns



CHAPTER 4.2 · LISTINGS 4.1–4.2

One Problem, One Pluggable Generator

Swap only the decoding function, so every comparison is fair.

RUNNING EXAMPLE · MATH-500

Half the value of 3x − 9 is x + 37. What is the value of x?

QWEN3 0.6B BASE, GREEDY

```
\boxed{20}
```

Wrong: the answer is 83.

Run on the CPU first: GPU kernels can change sampled results.

```
def generate_text_stream_concat_flex(model, tokenizer, prompt,
        device, max_new_tokens, verbose=False,
        generate_func=None, **generate_kwargs):
    if generate_func is None: # default: greedy (Ch. 2)
        generate_func = generate_text_basic_stream_cache
    input_ids = torch.tensor(tokenizer.encode(prompt),
        device=device).unsqueeze(0)
    generated_ids = []
    for token in generate_func(model=model, token_ids=input_ids,
            max_new_tokens=max_new_tokens,
            eos_token_id=tokenizer.eos_token_id, **generate_kwargs):
        generated_ids.append(token.squeeze(0).item())
    return tokenizer.decode(generated_ids)
```

CHAPTER 4.3

Chain-of-Thought Prompting in One Line

Ask for the steps, and the 0.6B base model finds 83.

THE WHOLE METHOD

```
prompt_cot = prompt + " \n\nExplain step by step."
response_cot = generate_text_stream_concat_flex(
    model, tokenizer, prompt_cot, device,
    max_new_tokens=2048, verbose=True)
```

RESPONSE (ABRIDGED)

```
### Step 3: Solve for x
Subtract 2x from both sides: 3x - 2x - 9 = 74
x - 9 = 74, so x = 74 + 9 = 83
### Final Answer: \boxed{83}
```

COST

Every reasoning token is one more forward pass: latency, compute and API cost all grow.

OVERTHINKING

On simple problems, CoT can add erroneous steps that mislead the model.

ONLY FOR SOME MODELS

Reasoning models already explain their answers and gain nothing from the instruction.

CHAPTER 4.3 · BOX

When Chain-of-Thought Helps

No new knowledge — a different use of the knowledge the model already has.

MORE CHANCES TO SELF-CORRECT

Writing out the steps gives the model more opportunities to notice and fix its own mistakes.

MATCHES THE TRAINING DATA

Math and logic corpora are full of worked solutions; asking for steps puts the model on familiar ground.

MORE SERIAL COMPUTATION

Each token is one more pass through the network: intermediate steps let a fixed-size transformer solve problems that need more sequential depth than one pass.

Extension: Feng et al., NeurIPS 2023; Merrill & Sabharwal, ICLR 2024

Not a guarantee: chains can be fluent and wrong, and easy problems can be overthought.

CHAPTER 4.4.1 · FIGURES 4.5–4.6

From Logits to the Next Token

Self-consistency needs diverse samples, so we open up the decoding step.

STEP 3.1

Get next-token scores

`model(ids)[:, -1]`
`shape [1, 151936]`

→

STEP 3.2

Rescale the scores

`logits / T`
`(temperature)`

→

STEP 3.3

Convert to probabilities

`torch.softmax`

→

STEP 3.4

Sample the next token

`torch.multinomial`
`(greedy: argmax)`

“The capital of Germany is” → token IDs [785, 6722, 315, 9856, 374] → the largest of 151,936 logits sits at index 19846 → “␣Berlin”.

Top-p filtering (step 4, section 4.5) will slot in between steps 3.3 and 3.4.

CHAPTER 4.4.2–4.4.3 · LISTINGS 4.4–4.6

Temperature Rescales the Logits

Divide by T before the softmax: T < 1 sharpens, T > 1 flattens.

$$p_i(T) = \exp(z_i / T) / \Sigma_j \exp(z_j / T)$$

- T = 1: the model's own distribution
- $T \to 0^+$: all mass on the largest logit, i.e. greedy decoding
- $T \to \infty$: uniform over all 151,936 tokens
- Entropy never decreases as T grows

```
def scale_logits_by_temperature(logits, temperature):
    if temperature <= 0:
        raise ValueError("Temperature must be positive")
    return logits / temperature
```

T = 0.5

0.3398

probability of "␣Berlin" after "The capital of Germany is"

T = 5

0.0003

the same token, after flattening — still the most likely one

CHAPTER 4.4.3 AND 4.5.2 · LISTING 4.7

What 1,000 Draws Look Like

Next token after “The capital of Germany is”, sampled 1,000 times.

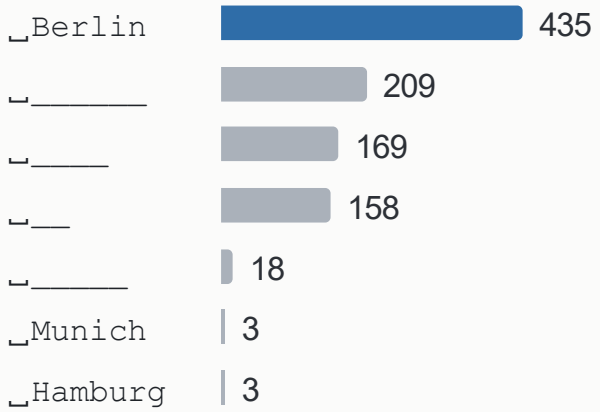
T = 5

No token is drawn more than twice, and every one is nonsense:

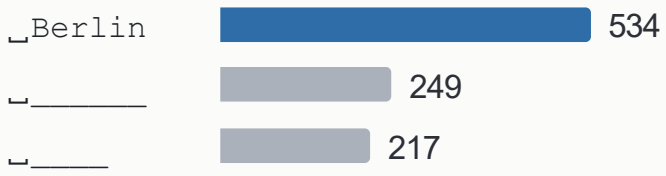
```
}  ␣</  ␣represent
␣Inf  ()*  ␣beside  ␣Kob
```

p(␣Berlin) = 0.0003

T = 0.35



T = 0.35 + TOP-P 0.8



The nucleus keeps three tokens; ␣___, ␣_____, ␣Munich and ␣Hamburg are never drawn.

CHAPTER 4.4.4 · LISTING 4.8

Temperature Inside the Generation Loop

Same KV-cached loop as Chapter 2; only the choice of token changes.

```
def generate_text_temp_stream_cache(model, token_ids, max_new_tokens,
        eos_token_id=None, temperature=0.):
    cache = KVCache(n_layers=model.cfg["n_layers"])
    out = model(token_ids, cache=cache)[:, -1]
    for _ in range(max_new_tokens):
        if temperature is None or temperature == 0.0:
            next_token = torch.argmax(out, dim=-1, keepdim=True)
        else: # NEW: steps 3.2-3.4
            logits = scale_logits_by_temperature(out, temperature)
            probas = torch.softmax(logits, dim=-1)
            next_token = torch.multinomial(probas.cpu(), num_samples=1)
            next_token = next_token.to(token_ids.device)
        if (eos_token_id is not None
                and torch.all(next_token == eos_token_id)):
            break
        yield next_token
        out = model(next_token, cache=cache)[:, -1]
```

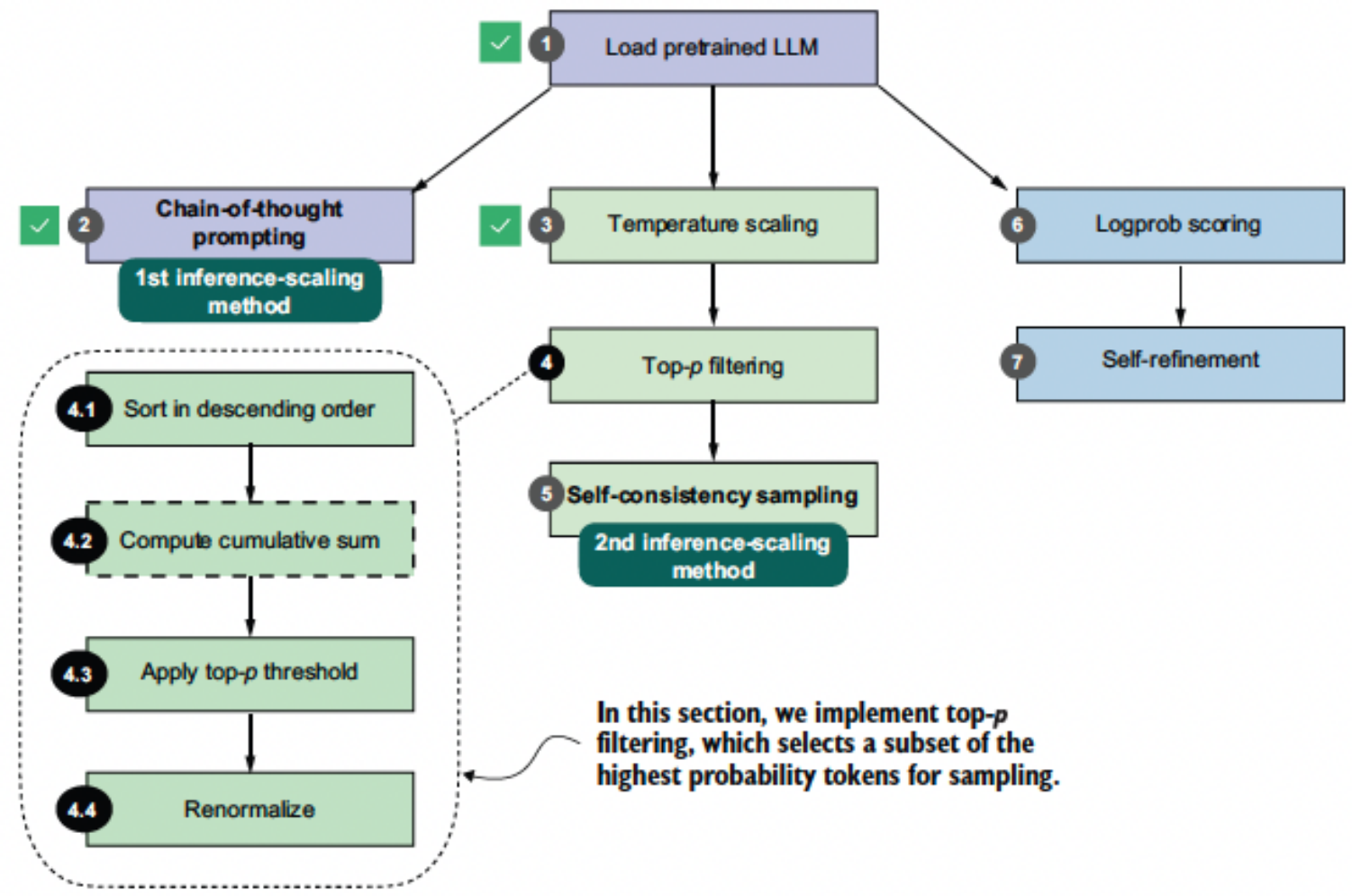
T = 0 or None falls back to argmax, so one function covers greedy and sampling.

Sampling runs on the CPU for reproducible draws.

Still wrong: T = 1.1 on the running example gives x = 90/7. Diversity is not accuracy.

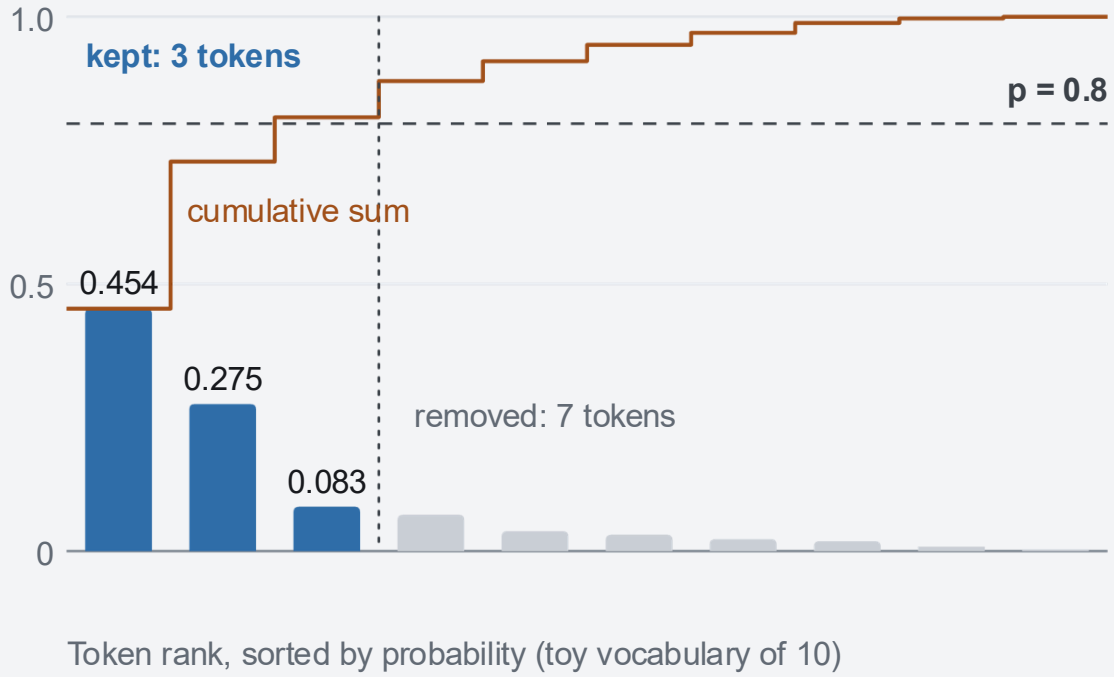
CHAPTER 4.5.1 · LISTINGS 4.9–4.14

Top-p Keeps the Smallest Sufficient Set



Top-p Keeps the Smallest Sufficient Set

Nucleus sampling: cut off the unreliable tail, then renormalize what is left.



Keep the smallest set of top tokens whose probabilities sum to at least p; zero the rest; renormalize.

TWO CUT-OFF RULES, P = 0.8

`cumsum <= top_p`

keeps 2 tokens: it stops below p

`(cumsum - sorted_probas) < top_p`

keeps 3: includes the token that crosses p (used from here on)

Renormalized kept mass: 0.5589, 0.3390, 0.1021

CHAPTER 4.5.2 · LISTINGS 4.15–4.16

Top-p Between Softmax and Sampling

One new line in the loop: probas = top_p_filter(probas, top_p).

```
def top_p_filter(probas, top_p):
    if top_p is None or top_p >= 1.0:
        return probas
    sorted_p, sorted_idx = torch.sort(probas, dim=1,
                                      descending=True)
    cum_p = torch.cumsum(sorted_p, dim=1)
    keep = (cum_p - sorted_p) < top_p # mass before each token
    keep[:, 0] = True # always keep the top token
    kept = torch.where(keep, sorted_p, torch.zeros_like(sorted_p))
    filtered = torch.zeros_like(probas).scatter(
        1, sorted_idx, kept) # back to vocabulary order
    denom = filtered.sum(dim=1, keepdim=True).clamp_min(1e-12)
    return filtered / denom
```

TRUNCATION RULES

Top-k: keep the k most likely tokens; a fixed count.

Top-p: smallest set with mass ≥ p; the count adapts.

Min-p: keep p ≥ p_base · p_max; scales with confidence.

Running example with T = 0.5 and top-p = 0.8: \boxed{18}. Still wrong — diversity is the input to voting.

CHAPTER 4.6 · FIGURE 4.18 · LISTING 4.17

Self-Consistency: Sample, Extract, Vote

Wang et al. (ICLR 2023) from Session 5, now as code on our harness.

```
def self_consistency_vote(model, tokenizer, prompt, device,
        num_samples=10, temperature=0.8, top_p=0.9):
    short_answers = []
    for i in range(num_samples): # 1) sample
        answer = generate_text_stream_concat_flex(
            model, tokenizer, prompt, device, max_new_tokens=2048,
            generate_func=generate_text_top_p_stream_cache,
            temperature=temperature, top_p=top_p)
        short_answers.append( # 2) extract
            extract_final_candidate(answer, fallback="number_then_full"))
    mc = Counter(short_answers).most_common() # 3) vote
    winners = [s for s, f in mc if f == mc[0][1]]
    return winners[0] if len(winners) == 1 else None # tie: None
```

SEED 123, N = 5

```
Sample 1 → 83
Sample 2 → 22
Sample 3 → 54
Sample 4 → 83
Sample 5 → 61
```

Vote: 83 (2 of 5)

With “Explain step by step.” added, all five samples say 83.

CHAPTER 4.6 · ANALYSIS

Why Voting Works, and When It Fails

The vote is right when the correct answer is the mode — not when p > ½.

$$\hat{a} = \operatorname{argmax}_a \Sigma_r \, p(r, a \mid x)$$

Self-consistency approximates marginalizing over reasoning paths r: correct paths tend to agree on the answer a, wrong paths scatter.

With n samples, a plurality vote converges to the mode of p(a | x). In the book's run, 83 repeats; 22, 54 and 61 do not.

Voting needs short, comparable answers — the gap Chapter 5 addresses.

A MODEL THAT IS RIGHT 40% OF THE TIME

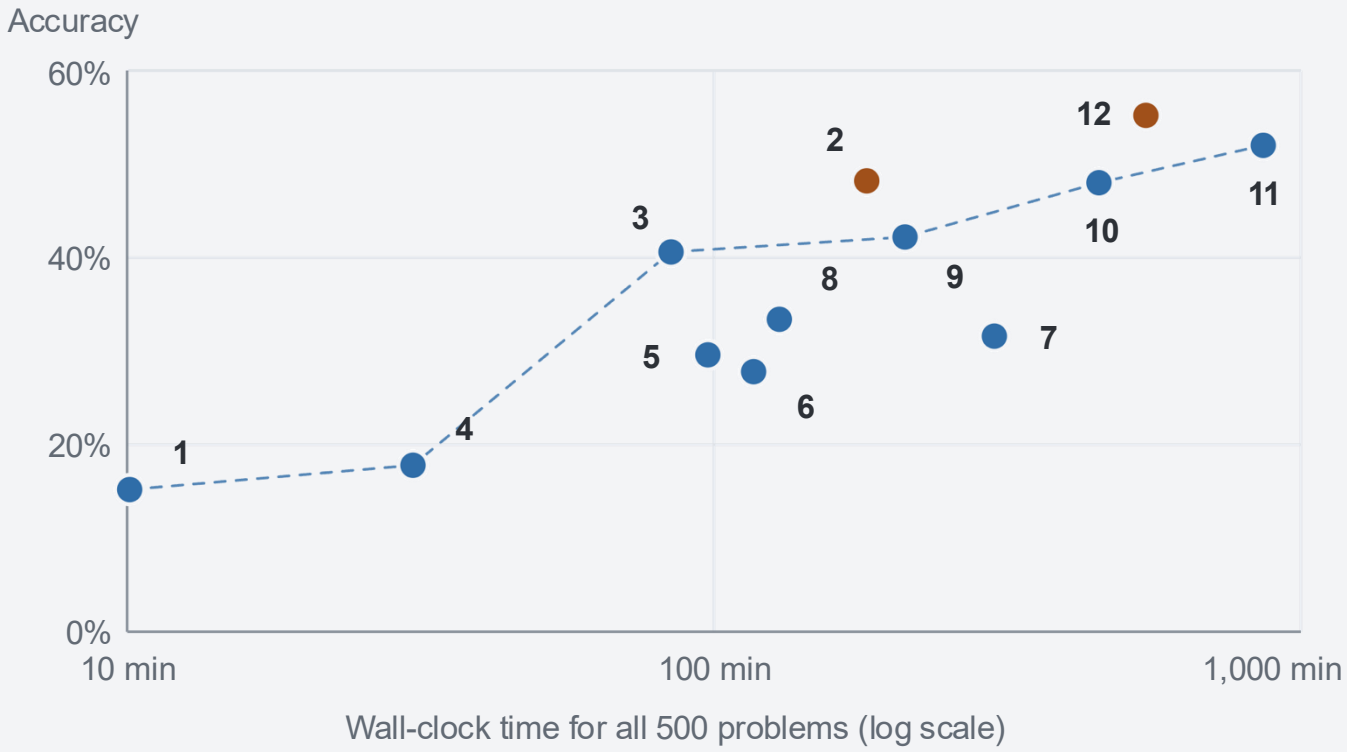
Samples n	Errors scattered	One systematic error
1	0.40	0.40
5	0.58	0.32
15	0.83	0.21
∞	→ 1	→ 0

Plurality-vote accuracy. Scattered: five wrong answers at 0.12 each. Systematic: one wrong answer at 0.6. Ties broken at random.

CHAPTER 4.6 · TABLE 4.1, REPLOTTED

Accuracy Against Compute on MATH-500

Twelve configurations on all 500 problems; sampling uses T = 0.9 and top-p = 0.9.



Base · Reasoning · Base frontier

1 Greedy, base · 15.2% · 10 min
2 Greedy, reasoning · 48.2% · 182 min
3 CoT · 40.6% · 85 min
4 Temperature + top-p · 17.8% · 31 min
5 SC n=3 · 29.6% · 98 min
6 SC n=5 · 27.8% · 117 min
7 SC n=10 · 31.6% · 300 min
8 Top-p + CoT · 33.4% · 129 min
9 SC n=3 + CoT · 42.2% · 212 min
10 SC n=5 + CoT · 48.0% · 453 min
11 SC n=10 + CoT · 52.0% · 863 min
12 SC n=3 + CoT, reasoning · 55.2% · 544 min

CHAPTER 4.6 · TABLE 4.1

Five Readings of Table 4.1

Better accuracy for more compute — but not every extra token is equally useful.

15.2 → 40.6%	CoT alone gives the largest single gain (2.7×) for 8.4× the time.
17.8%	Temperature and top-p alone barely move accuracy: they control diversity; by themselves they are not a scaling method.
29.6 → 31.6%	Voting without CoT plateaus: n = 3 to n = 10 adds 2 points for 3× the time.
33.4% < 40.6%	Sampling hurts a single CoT chain, yet voting over sampled chains beats greedy CoT: 52.0% at n = 10, for 85× the baseline time.
48.2 → 55.2%	The reasoning model gains too: inference scaling stacks on top of training.

EXTENSION · RECENT LITERATURE

Spending a Test-Time Budget Well

Two results that frame the methods in this block.

SNELL ET AL., 2024 · COMPUTE-OPTIMAL SCALING

The best use of test-time compute depends on prompt difficulty: sequential revision versus parallel search against a verifier.

> 4×
more efficient than a best-of-N baseline

14×
larger model beaten at matched FLOPs

BROWN ET AL., 2024 · LARGE LANGUAGE MONKEYS

Coverage, the chance that at least one sample is correct, keeps rising over four orders of magnitude of samples.

15.9 → 56%
SWE-bench Lite, 1 vs. 250 samples (DeepSeek-Coder-V2-Instruct)

Plateau
for voting and reward models beyond several hundred samples

For our harness: generating more answers is easy; choosing among them is the bottleneck. Chapter 5 builds scorers; Chapter 6 turns the verifier into a reward.

Block I in Four Lines

1. Inference-time scaling buys accuracy with compute at answer time; the weights do not change.
2. Chain-of-thought prompting makes answers longer: 15.2% → 40.6% on MATH-500 for the base model.
3. Temperature and top-p turn the decoder into a sampler. They control diversity, not accuracy.
4. Self-consistency votes over sampled answers; with CoT it reaches 52.0% at n = 10, for 85× the time.

BLOCK II · CHAPTER 5

Inference-Time Scaling via Self-Refinement

Score an answer, critique it, revise it — and keep the revision only if it scores at least as well.

CHAPTER 5.1 · FIGURES 5.2–5.3

From Voting to Refining

Voting needs comparable answers; refinement improves one answer at a time.

SELF-CONSISTENCY · BLOCK I

Many answers in parallel; return the most frequent. Needs short final answers that can be compared.

SELF-REFINEMENT · THIS BLOCK

One answer, improved sequentially: draft, critique, revision. Works on free-form outputs too.

ROAD MAP OF THE CHAPTER (FIGURE 5.3)



CHAPTER 5.2 · LISTING 5.2

Two Correct Answers: Which Is Better?

Same CoT prompt, T = 0.9 and top-p = 0.9: two seeds, two styles, one answer.

SEED 0 · RESPONSE 1

```
The problem states that half the value of 3x-9 is
equal to x+37. We need to find the value of x.
### Step 2: Translate the problem into an equation …
\boxed{83}
```

534 tokens 1,419 characters

SEED 3 · RESPONSE 2

```
We start with the given equation:
½ × (3x − 9) = x + 37
…
Final Answer: \boxed{83}
```

231 tokens 533 characters

"Better" depends on clarity, usefulness and the correctness of intermediate steps: a human judgment, which Block III will learn from data. When quality is comparable, the shorter answer is cheaper.

CHAPTER 5.3 · FIGURE 5.5

Scoring Without a Ground Truth

At inference time there is no reference answer, so we need a scorer, not a verifier.

RULE-BASED (HEURISTIC)

Hand-written rules: is there a \boxed{} answer? Is it short? Cheap and transparent, but easy to game.

Section 5.3

LLM-AS-A-JUDGE

A second model rates or critiques the answer. Flexible, but it costs another model call and inherits that model's biases.

Appendix F; DeepSeekMath-V2

MODEL CONFIDENCE

The model's own probability of its answer tokens. One forward pass; measures how expected an answer is, not whether it is true.

Sections 5.4–5.6

Verifier (Chapter 3): compares with a known answer, for evaluation. **Scorer**: ranks candidates without one, so it can run at deployment.

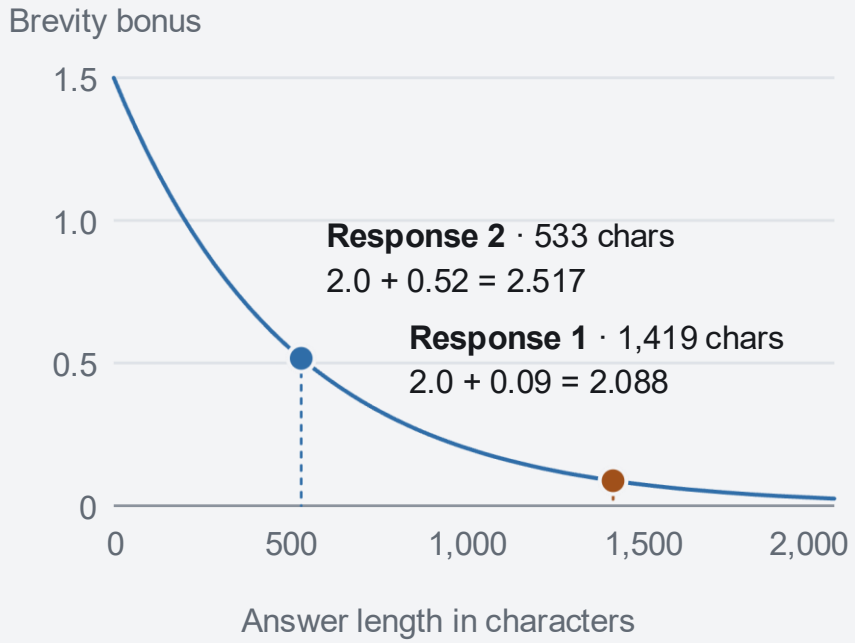
CHAPTER 5.3 · LISTINGS 5.3–5.4

A Rule-Based Scorer

Reward an extractable answer first; break ties by brevity.

score = 2.0 · [boxed answer] or 1.0 · [number only]
+ 1.5 · exp(−characters / 500)

```
def heuristic_score(answer, prompt=None, brevity_bonus=500.0,
                    boxed_bonus=2.0, extract_bonus=1.0):
    score = 0.0
    if extract_final_candidate(answer, fallback="none"):
        score += boxed_bonus # clean \boxed{} answer
    elif extract_final_candidate(answer, fallback="number_only"):
        score += extract_bonus # at least a number
    return score + 1.5 * math.exp(-len(answer) / brevity_bonus)
```



CHAPTER 5.4 · FIGURES 5.9–5.11

The Probability of a Whole Sequence

Chain rule: multiply each token's probability given everything before it.

$$p(x_2, \ldots, x_n \mid x_1) = \prod_{t=2}^{n} p(x_t \mid x_1, \ldots, x_{t-1})$$

Next token, given the prefix	"… is Berlin"	"… is Bridge"
␣capital	6.15×10^{-5}	6.15×10^{-5}
␣of	0.465	0.465
␣Germany	0.0167	0.0167
␣is	0.738	0.738
␣Berlin / ␣Bridge	0.169	2.98×10^{-7}
Joint probability	5.94×10^{-8}	1.05×10^{-13}

TINY NUMBERS

Both joint probabilities are near zero: multiplying many numbers below 1 heads toward underflow on long answers.

CONFIDENCE ≠ CORRECTNESS

A high probability means the answer fits learned patterns well, not that it is true.

CHAPTER 5.4 · BOX

Scoring Is Not Generating

We read probabilities off a finished answer; no token is chosen.

SCORING · THIS CHAPTER

Retrospective. The whole answer already exists; one forward pass returns $p(x_t \mid x_1 \ldots x_{t-1})$ at every position.

GENERATION · CHAPTER 4

Prospective. Greedy, temperature and top-p commit to one token per step, and each choice becomes the next input.

THREE CONSEQUENCES

- Greedy decoding need not find the most probable sequence: a locally best token can lead somewhere unlikely.
- Temperature and top-p move samples even further from the most probable sequence.
- Even the most probable sequence need not be correct or useful — it is only what the model prefers.

CHAPTER 5.4 · LISTING 5.5 · FIGURE 5.12

Reading Off Next-Token Probabilities

Shift by one: the output at position t scores the token at position t + 1.

```
@torch.inference_mode()
def calc_next_token_probas(model, tokenizer, prompt, device):
    token_ids = torch.tensor(tokenizer.encode(prompt), device=device)
    logits = model(token_ids.unsqueeze(0)).squeeze(0) # [T, V]
    all_probas = torch.softmax(logits, dim=-1)
    t_idx = torch.arange(0, token_ids.shape[0] - 1, device=device)
    next_ids = token_ids[1:] # targets: inputs shifted by one
    next_token_probas = all_probas[t_idx, next_ids]
    return next_token_probas, torch.prod(next_token_probas)
```

TOY EXAMPLE · FIGURE 5.12

```
"Some example text"
token_ids = [2, 1, 3]
t_idx     = [0, 1]
next_ids  = [1, 3]
p[0, 1] = 0.095
p[1, 3] = 0.060
product = 0.0057
```

Same gather as the pretraining loss: $-\Sigma \log p(\text{next token})$ is the cross-entropy. Chapter 6 reuses it for the policy's log-probabilities.

CHAPTER 5.5 · LISTINGS 5.6–5.7 · FIGURE 5.16

Log Probabilities: Products Become Sums

Same ranking, no underflow — the standard currency of machine learning.

$$\log p(x_2 \dots x_n \mid x_1) = \Sigma_{t=2}^{n} \log p(x_t \mid x_1 \dots x_{t-1})$$

Next token	"… is Berlin"	"… is Bridge"
␣capital	−9.6875	−9.6875
␣of	−0.7695	−0.7695
␣Germany	−4.0938	−4.0938
␣is	−0.3008	−0.3008
␣Berlin / ␣Bridge	−1.7812	−15.0000
Sum (joint log-probability)	−16.6250	−29.8750

TWO CODE CHANGES

```
torch.softmax → torch.log_softmax
torch.prod    → torch.sum
```

Mixing them runs, but is mathematically wrong.

WHY LOG SPACE

A 500-token answer at p = 0.5 per token has probability $2^{-500} \approx 3 \times 10^{-151}$: zero in fp32 or bf16. Its log, −346.6, is an ordinary number.

CHAPTER 5.6 · LISTING 5.8 · FIGURE 5.18

Average Log-Probability of the Answer

Score only the answer tokens, and average so that lengths are comparable.

$$s(y \mid x) = (1 / |y|) \cdot \Sigma_t \log p(y_t \mid x, y_1 \ldots y_{t-1})$$

The per-token log-likelihood of the answer; exp(−s) is its perplexity.

```
def avg_logprob_answer(model, tokenizer, prompt, answer,
                       device="cpu"):
    prompt_ids = tokenizer.encode(prompt)
    answer_ids = tokenizer.encode(answer)
    full_ids = torch.tensor(prompt_ids + answer_ids, device=device)
    logits = model(full_ids.unsqueeze(0)).squeeze(0)
    logprobs = torch.log_softmax(logits, dim=-1)
    start, end = len(prompt_ids) - 1, full_ids.shape[0] - 1
    t_idx = torch.arange(start, end, device=device)
    next_tokens = full_ids[start + 1 : end + 1]
    return torch.mean(logprobs[t_idx, next_tokens])
```

“WHAT IS THE CAPITAL OF GERMANY?”

“ The capital of Germany is Berlin.”

−0.2041

“ The capital of Germany is Bridge.”

−3.8906

Two changes from Listing 5.7 (highlighted): drop the prompt tokens, and take the mean instead of the sum.

CHAPTER 5.6 · DISCUSSION

Confidence Is Not Correctness

Log-probability measures how expected an answer looks, not whether it is true.

CONFIDENT · CORRECT

High score, right answer: the scorer agrees with the truth.

CONFIDENT · WRONG

Fluent, familiar and wrong. The scorer prefers it: the blind spot.

UNSURE · CORRECT

An unusual but valid derivation gets penalized.

UNSURE · WRONG

Low score, wrong answer: the scorer agrees with the truth again.

LENGTH EFFECTS

The summed log-probability favors short answers: every token adds a negative term.

The mean removes that bias but rewards long, predictable stretches such as boilerplate or repetition.

Use it as one signal among several — for example, as a tiebreaker.

CHAPTER 5.7 · FIGURE 5.20 · LISTINGS 5.9–5.11

Self-Refinement by Hand

Three calls to the same model: answer, critique, revise (T = 0.7, top-p = 0.9).

1 · INITIAL ANSWER

Prompt

```
render_prompt(question)
```

Output

```
\boxed{18}
```

Wrong

2 · CRITIQUE

Prompt

```
You are a meticulous reviewer. Identify logical errors, missing steps, or arithmetic mistakes. … Question: {q} Draft answer: {draft} Write a short critique and bullet-point fix plan. Critique:
```

Output

"… half of 3x−9 should be (3x−9)/2 … Fix plan: ½(3x−9) = x+37 … x = 83." Partly false, but the plan is right.

3 · REVISION

Prompt

```
Revise the answer using the critique. Keep it concise and end with a final boxed result: \boxed{ANSWER} … Previous answer … Critique … Revised answer:
```

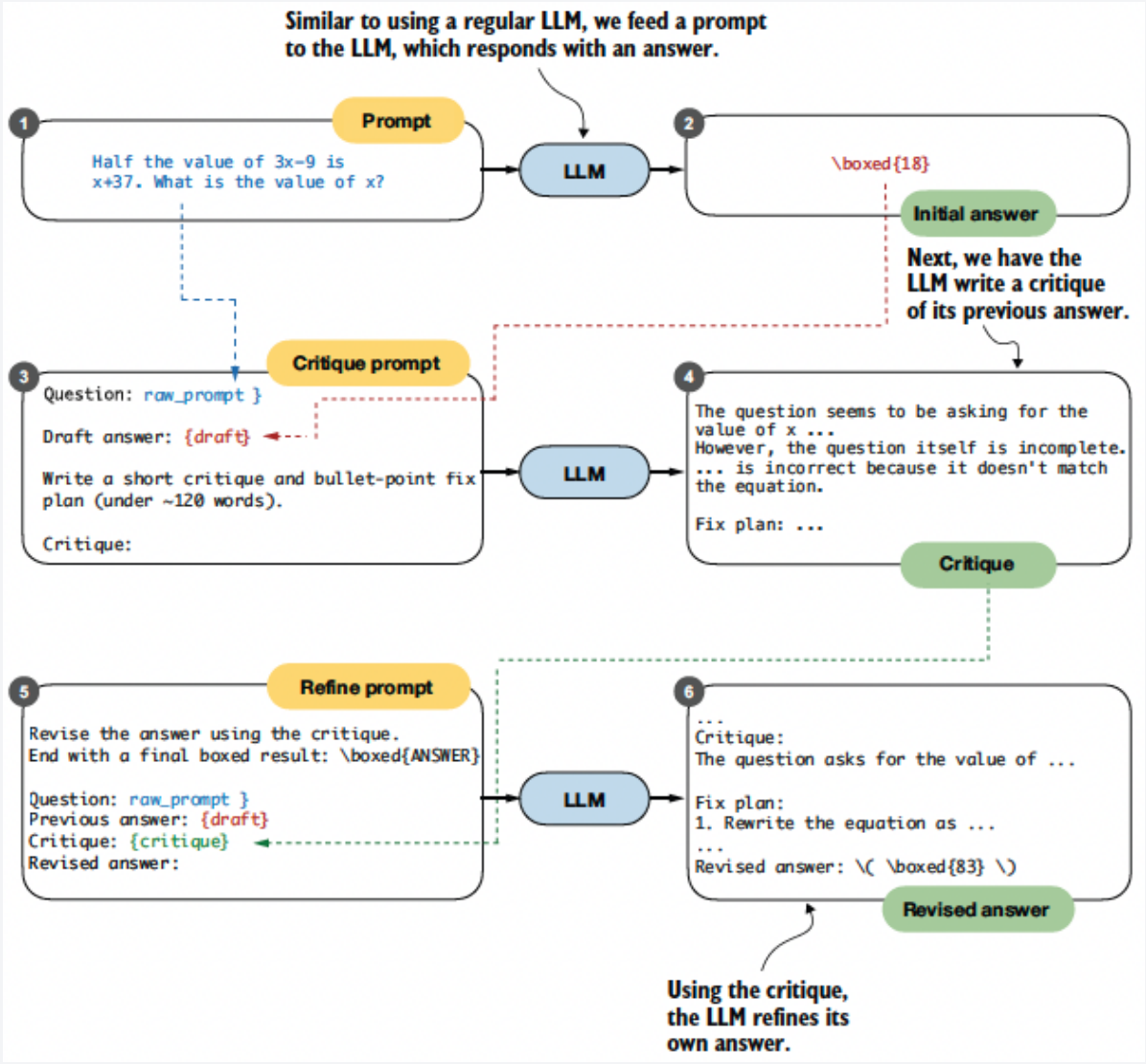
Output

```
\boxed{83}
```

Correct

CHAPTER 5.7 · FIGURE 5.20 · LISTINGS 5.9–5.11

Self-Refinement by Hand



The Self-Refinement Loop

Repeat critique and revision; accept a revision only if its score does not drop.

```
def self_refinement_loop(model, tokenizer, raw_prompt, device,
                         iterations=2, score_fn=None):
    prompt = render_prompt(raw_prompt)
    def score(a):
        return score_fn(answer=a, prompt=prompt) if score_fn else 0.0
    current = generate(prompt) # draft
    current_score = score(current)
    for it in range(iterations):
        critique = generate(make_critique_prompt(raw_prompt, current))
        revised = generate(make_refine_prompt(raw_prompt, current,
                                              critique))
        revised_score = score(revised)
        if revised_score >= current_score: # accept if not worse
            current, current_score = revised, revised_score
    return current
```

TRACE, LOGPROB SCORER

1 · 10 → 83

score −0.855 → −0.226, accepted

2 · 83 → 83

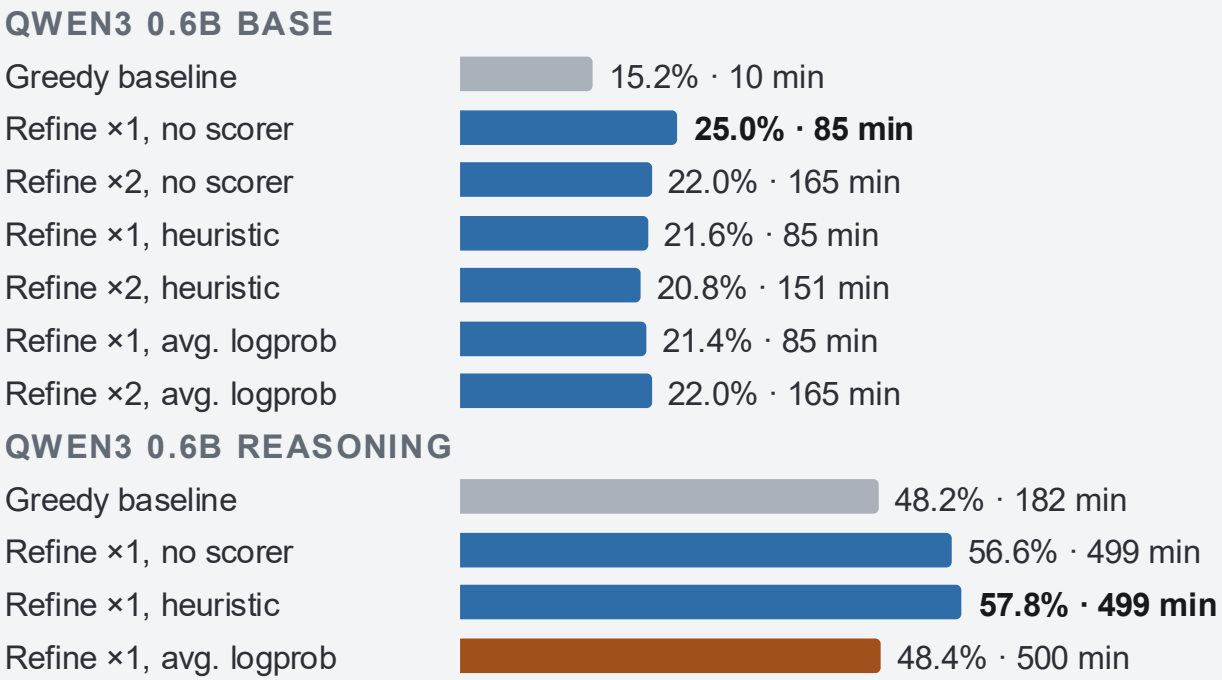
score −0.226 → −1.320, rejected

Seed 1, T = 0.7, top-p = 0.9. generate(…) is the Chapter 4 top-p sampler; the scorer is avg_logprob_answer wrapped with functools.partial.

CHAPTER 5.8 · TABLE 5.1

Self-Refinement on MATH-500

Gains are real but modest — and the scorer can hurt.



The best base result uses no scorer (25.0%); a second round lowers it to 22.0%.

For the reasoning model, the logprob scorer erases the gain (48.4% vs. 56.6–57.8%): it accepts confident mistakes.

At similar cost, voting wins: self-consistency n = 3 reaches 29.6% in 98 min (Table 4.1).

EXTENSION · RECENT LITERATURE

Can a Model Correct Itself?

It depends on where the feedback comes from.

MADAAN ET AL. · NEURIPS 2023

Self-Refine

The same LLM writes feedback and refines, with no training: about 20% absolute average gain over 7 tasks (GPT-3.5, ChatGPT, GPT-4), mostly open-ended ones.

HUANG ET AL. · ICLR 2024

Cannot self-correct reasoning yet

Without external feedback, reasoning accuracy often fails to improve, and sometimes drops, after self-correction.

SHAO ET AL. · 2025

DeepSeekMath-V2

Train a strong critic, then train the solver to refine against it: gold-level results in several math competitions.

Table 5.1 sits in between: intrinsic refinement of a 0.6B model helps a little. Learned or external feedback is the lever.

EXTENSION · RECENT LITERATURE

Confidence Meets Voting

Blocks I and II combined: filter the votes by the model's own confidence.

1 · Generate many reasoning traces in parallel (Block I)

2 · Score each trace by token-level confidence from its log-probabilities (Block II)

3 · Drop low-confidence traces, during generation (stop early) or after it

4 · Vote among the surviving traces

99.9%

on AIME 2025 with DeepConf@512, at best

−84.7%

generated tokens vs. full parallel thinking, at best

Fu et al., 2025, "Deep Think with Confidence"; open models (Qwen3, GPT-OSS)

Four Inference-Time Methods

What each one needs, what it costs, and how it fails.

Method	Needs	Cost per question	Book, MATH-500	Main failure mode
CoT prompting	an instruction in the prompt	one longer output (≈ 8× time)	15.2% → 40.6% (base)	overthinking easy problems
Self-consistency	short, comparable final answers	n full generations	52.0% (n = 10, with CoT)	a systematic error wins the vote
Best-of-N	a scorer	n generations plus scoring	Exercises 5.2 and 5.4	picks what the scorer likes
Self-refinement	critique and refine prompts (+ scorer)	about 3 calls per round	25.0% base; 57.8% reasoning	accepting confident mistakes

THINKING PROBLEMS · BLOCK II · 8 MIN

Thinking Problems

Work in pairs; we will discuss two of the three.

Q1 · SUM OR MEAN?

Show that the summed log-probability always prefers a strict prefix of an answer to the full answer. Then construct two answers for which the mean log-probability ranks a wrong answer above a right one.

Q2 · WHY DID LOGPROB HURT?

With the reasoning model, refinement + heuristic reaches 57.8%, but refinement + average log-probability only 48.4%. Propose two mechanisms, and an experiment that tells them apart.

Q3 · A MARKOV CHAIN

Each round turns a wrong answer right with probability a, and a right answer wrong with probability b. Find the accuracy after k rounds and its limit. When does a second round help? Compare rows 2–3 of Table 5.1.

Block II in Four Lines

1. Scorers rank answers without a ground truth: rules (format, brevity), the model's own probabilities, or another model.

2. A sequence log-probability is a sum of next-token log-probs from one forward pass; averaging makes lengths comparable.

3. Self-refinement is draft → critique → revision, optionally keeping only revisions that score at least as well.

4. On MATH-500 it helps modestly (15.2 → 25.0% base, 48.2 → 57.8% reasoning); a confidence scorer can accept confident mistakes.

BLOCK III · PREFERENCE TUNING · 50 MIN

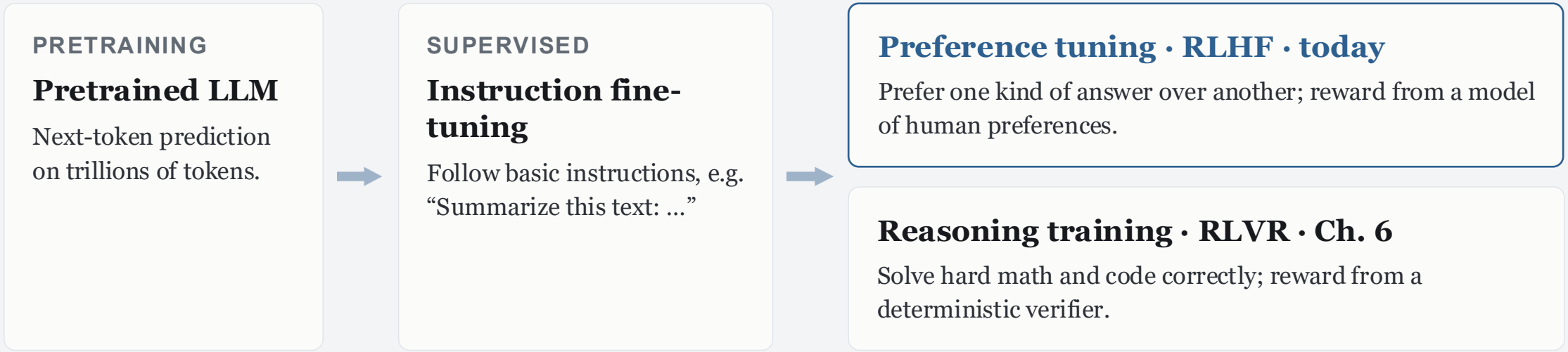
Reinforcement Learning from Human Feedback

When no verifier exists, learn a reward from human comparisons — then optimize it on a leash.

CHAPTER 6.1 · FIGURE 6.3

Where Preference Tuning Sits

Two RL stages can follow instruction tuning; they differ in where the reward comes from.



The order varies: pipelines interleave the two stages, and DeepSeek-R1-Zero applied RLVR directly to the base model.

MOTIVATION

Why Not More Supervised Fine-Tuning?

Judging is easier than demonstrating, and the target is a whole response.

COMPARISONS ARE CHEAPER

Writing an ideal answer is hard; picking the better of two is fast and more consistent.

NO SINGLE RIGHT ANSWER

Likelihood training treats every deviation from a demonstration as equally wrong.

A SEQUENCE-LEVEL OBJECTIVE

RL optimizes a score of the whole response — helpfulness, tone, safety — not the next token.

LEARNS FROM ITS OWN OUTPUTS

On-policy samples expose the model's own mistakes, which fixed demonstrations never show.

RLHF asks people which of two answers is better, then learns to generalize their judgments.

LITERATURE · 2017–2025

A Short History of RLHF

From simulated robots to chat assistants to reasoning models.

2017

Christiano et al.

Deep RL from human preferences on Atari and MuJoCo: feedback on under 1% of interactions, about an hour of human time.

2019

Ziegler et al.

Fine-tunes GPT-2 from human preferences, with a KL penalty to the pretrained model.

2020

Stiennon et al.

Summarization: RLHF beats much larger supervised models and the human reference summaries.

2022

Ouyang et al.; Bai et al.

InstructGPT: SFT, reward model, PPO — the recipe behind ChatGPT. Anthropic's HH-RLHF; Constitutional AI uses AI feedback.

2023

Rafailov et al.

DPO: optimize preferences directly with a classification loss; no RL loop.

2025

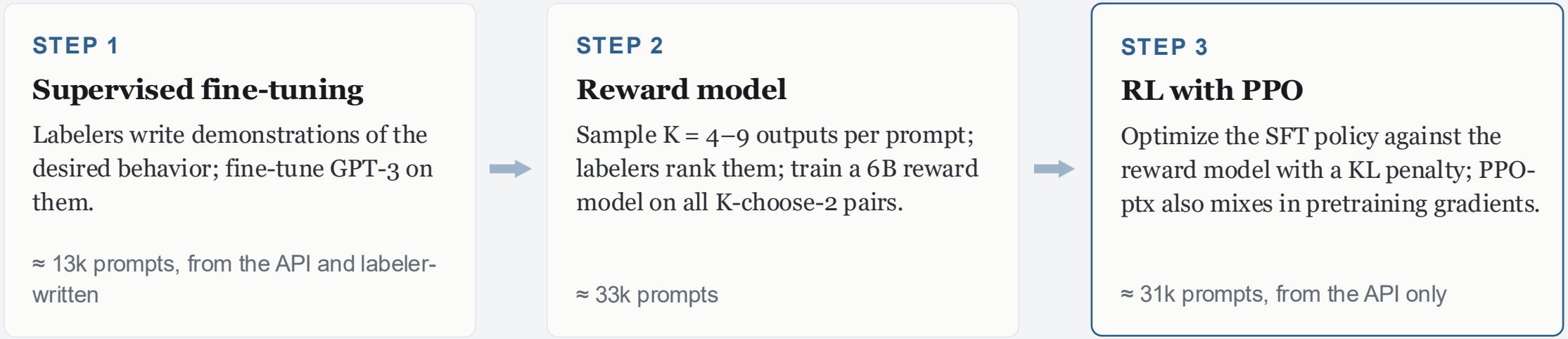
DeepSeek-R1

Verifiable rewards drive reasoning; preference rewards remain for helpfulness and harmlessness.

OUYANG ET AL., 2022 · FIGURE 2

The InstructGPT Recipe

Three steps; steps 2 and 3 can be iterated.



OUYANG ET AL., 2022 · RESULTS

What RLHF Bought InstructGPT

Human preference ratings on the API prompt distribution.

1.3B > 175B

Outputs of the 1.3B InstructGPT model were preferred to those of 175B GPT-3, with over 100× fewer parameters.

85 ± 3%

175B InstructGPT preferred over 175B GPT-3 (71 ± 4% over few-shot prompted GPT-3).

≈ 2×

as often truthful and informative on TruthfulQA; 25% fewer toxic outputs when prompted to be respectful.

ALIGNMENT TAX

Regressions on SQuAD, DROP, HellaSwag and WMT translation. Mixing pretraining gradients into PPO (PPO-ptx) reduces them.

PREFERENCE DATA

Collecting Comparisons

A prompt, two or more responses, and a human choice.

EXAMPLE · BOOK, SECTION 6.1.1

“I am looking to buy a laptop for programming and everyday use. What should I consider?”

A · “You should consider the CPU, RAM, storage, GPU, screen resolution, battery life, keyboard quality, port selection, thermal design, and price.”

B · “For programming and everyday use, focus on a fast CPU, at least 16 GB of RAM, and an SSD with at least 256 GB storage. A comfortable keyboard and good battery life also matter. …”

Annotator prefers B: concrete and specific to the use case.

Rankings → pairs. Ranking K responses gives $K(K-1)/2$ comparisons: $K = 9$ yields 36 pairs from one prompt.

Batch by prompt. InstructGPT put all pairs from a prompt in one batch element: one reward-model pass per response, and no overfitting.

Noisy labels. Labelers agree only 73–77% of the time — a ceiling for any reward model’s accuracy.

REWARD MODEL · BRADLEY–TERRY

From Comparisons to a Scalar Reward

Model the probability of a preference, then fit it by maximum likelihood.

$$P(y^+ \succ y^- \mid x) = \sigma(\, r(x, y^+) - r(x, y^-)\,)$$

$$\text{Loss} = -E[\, \log \sigma(\, r(x, y^+) - r(x, y^-)\,)\,]$$

Architecture: the LLM with its vocabulary head replaced by a linear layer — one scalar, read at the last token.

Only differences matter: $r + c(x)$ fits equally well, so rewards are normalized before RL (InstructGPT: demonstrations average 0).

```
class RewardModel(torch.nn.Module): # illustrative
    def __init__(self, trunk, d_model):
        super().__init__()
        self.trunk = trunk # LLM without LM head
        self.head = torch.nn.Linear(d_model, 1)

    def forward(self, ids, last): # last: final token
        h = self.trunk(ids) # [B, T, d_model]
        h = h[torch.arange(ids.shape[0]), last]
        return self.head(h).squeeze(-1) # [B] rewards

def rm_loss(r_pos, r_neg): # Bradley-Terry NLL
    return -F.logsigmoid(r_pos - r_neg).mean()
```

RL FORMULATION

Text Generation as a Decision Process

One response is one episode; one token is one action.

RL term	In an LLM
State s_t	the prompt x and the tokens so far: $(x, y_1 \dots y_{t-1})$
Action a_t	the next token y_t — one of 151,936 for Qwen3
Policy $\pi(a_t \mid s_t)$	the LLM's next-token distribution
Transition	deterministic: append the token
Reward	r(x, y) from the reward model, at the end of the response (sparse)
Episode	one complete response, ended by EOS or a length limit

$\log \pi(y \mid x) = \Sigma_t \log \pi(y_t \mid x, y_1 \dots y_{t-1})$: the sequence log-probability from Block II.

RL FORMULATION · ZIEGLER ET AL., 2019 · OUYANG ET AL., 2022

The KL-Regularized Objective

Maximize the learned reward without drifting far from the reference model.

$$\max_{\pi} \; \mathbb{E}[\, r(x, y) \,] - \beta \cdot \mathrm{KL}(\, \pi(\cdot \mid x) \,\|\, \pi_{\text{ref}}(\cdot \mid x) \,), \quad x \sim D, \; y \sim \pi(\cdot \mid x)$$

PROTECTS THE REWARD MODEL

The reward model is accurate only near its training data; the penalty keeps samples there.

KEEPS WHAT WAS LEARNED

Fluency, knowledge and diversity of π_{ref}, the SFT model, are preserved.

ONE KNOB

β trades reward against drift: $\beta \to 0$ exploits the reward model; $\beta \to \infty$ returns π_{ref}.

In practice the penalty is per token: $r_t = -\beta \cdot \log[\, \pi(y_t \mid s_t) \,/\, \pi_{\text{ref}}(y_t \mid s_t) \,]$, plus $r(x, y)$ at the last token. InstructGPT's PPO-ptx adds $\gamma \cdot \mathbb{E}[\, \log \pi(\text{pretraining text}) \,]$.

POLICY GRADIENT · WILLIAMS, 1992

Policy Gradients in One Slide

Raise the log-probability of what scored well; lower what scored badly.

$$\nabla J(\theta) = E[\ \Sigma_t\ \nabla \log \pi(y_t \mid s_t) \cdot \hat{A}_t\ ]$$

$$\hat{A}_t = \text{return} - \text{baseline } b(s_t)$$

The baseline does not bias the gradient but cuts its variance. The choice of baseline separates the algorithms:

- **REINFORCE:** sequence reward minus a simple baseline
- **PPO:** a learned value model V(s), the critic
- **GRPO (Chapter 6):** the mean reward of several samples for the same prompt

```
logp = seq_logprob(policy, x, y) # Block II
loss = -(adv.detach() * logp).mean()
loss.backward()
```

INTUITION

Supervised fine-tuning on the model's own samples, each weighted by its advantage: positive → imitate more; negative → push away.

PPO · SCHULMAN ET AL., 2017

PPO: Clipped Policy Updates

Reuse each batch for several steps, but never trust a large change.

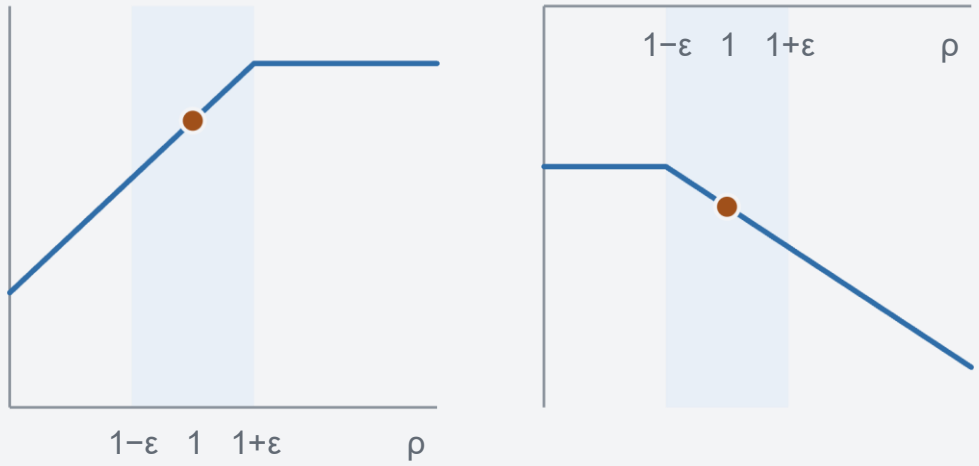
$$\rho_t = \pi(y_t \mid s_t) \,/\, \pi_old(y_t \mid s_t)$$

$$L_CLIP = E[\, \min(\, \rho_t \hat{A}_t,\ clip(\rho_t,\, 1 - \varepsilon,\, 1 + \varepsilon)\, \hat{A}_t \,) \,]$$

$$\text{GAE: } \hat{A}_t = \Sigma_l\, (\gamma\lambda)^l\, \delta_{t+l},\ \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

$\varepsilon \approx 0.2$. Outside the band the clipped term is flat, so one batch cannot push the policy far from π_old.

The full loss adds a value-function loss, often an entropy bonus.



L_CLIP for one token against the ratio ρ; the dot marks ρ = 1

PPO-BASED RLHF IN PRACTICE

Four Models, One Loop

Generation dominates the wall-clock time; four LLM-sized networks dominate memory.

1 · GENERATE

The policy π samples responses y for a batch of prompts x.

2 · SCORE

The reward model gives r(x, y); the reference π_ref gives per-token KL penalties.

3 · ESTIMATE

The value model $V(s_t)$ and GAE turn rewards into advantages $\hat{A}_t$.

4 · UPDATE

A few PPO epochs on minibatches; then $\pi_old \leftarrow \pi$, and repeat.

Policy π · trained

Value V · trained, initialized from the RM

Reward r · frozen

Reference π_ref · frozen SFT model

Dropping the value model is the key simplification of RLOO and GRPO (Chapter 6): the baseline comes from other samples of the same prompt.

FAILURE MODES

What Reward Hacking Looks Like

Each is a gap between what raters reward and what we actually want.

Failure	What happens	Evidence
Length bias	Longer answers score higher; part of the RLHF gain is verbosity	Singhal et al., 2024
Sycophancy	Agreeing with the user beats being right	Sharma et al., ICLR 2024
Over-refusal	Safety rewards teach refusing harmless requests	Röttger et al., 2024 (XSTest)
Less diversity	Outputs collapse toward a narrow mode	Kirk et al., ICLR 2024
Alignment tax	Regressions on standard NLP benchmarks	Ouyang et al., 2022
Worse calibration	Stated confidence stops matching accuracy	GPT-4 technical report, 2023

CASE STUDY · APRIL 2025

A Reward Signal That Favored Agreement

A production update to GPT-4o, rolled back within days.

25 APRIL

An update to GPT-4o ships; replies become markedly sycophantic.

27 APRIL

System-prompt changes as an emergency mitigation.

28 APRIL

Rollback to the previous model begins.

> *"… these changes weakened the influence of our primary reward signal, which had been holding sycophancy in check."*
>
> OpenAI, "Expanding on what we missed with sycophancy", May 2025

The changes included an extra reward signal from users' thumbs-up and thumbs-down data, which "can sometimes favor more agreeable responses." Lessons drawn: treat behavior issues as launch-blocking; distrust metrics that conflict with qualitative testing.

RAFAILOV ET AL. · NEURIPS 2023

DPO: The Same Objective, No RL Loop

Solve the KL-regularized problem in closed form, then fit the policy to preferences.

1 OPTIMAL POLICY OF THE KL-REGULARIZED OBJECTIVE

$$\pi^*(y \mid x) = \pi_{\text{ref}}(y \mid x) \cdot \exp(\, r(x, y) \,/\, \beta \,) \,/\, Z(x)$$

2 INVERT IT: THE REWARD IN TERMS OF THE POLICY

$$r(x, y) = \beta \cdot \log[\, \pi^*(y \mid x) \,/\, \pi_{\text{ref}}(y \mid x) \,] + \beta \cdot \log Z(x)$$

3 PLUG INTO BRADLEY–TERRY: Z(X) CANCELS IN THE DIFFERENCE

$$L_{\text{DPO}} = -E[\, \log \sigma(\, \beta \log \pi(y^+|x)/\pi_{\text{ref}}(y^+|x) - \beta \log \pi(y^-|x)/\pi_{\text{ref}}(y^-|x) \,) \,]$$

The policy is its own reward model: $\hat{r}(x, y) = \beta \cdot \log \pi(y \mid x) \,/\, \pi_{\text{ref}}(y \mid x)$.

DPO IN PRACTICE

DPO Is a Classification Loss

Four sequence log-probabilities per pair: the Block II function, run on two models.

```
def dpo_loss(pi_pos, pi_neg, ref_pos, ref_neg, beta=0.1):
    # each: summed log-prob of the response tokens, shape [B]
    logits = beta * ((pi_pos - ref_pos) - (pi_neg - ref_neg))
    return -F.logsigmoid(logits).mean()

# pi_*: policy, with gradients; ref_*: frozen reference
# computed under torch.no_grad(), usually the SFT model
```

$$\nabla L = -\beta \cdot \mathbb{E}\left[\sigma\left(\hat{r}(x, y^-) - \hat{r}(x, y^+) \right) \cdot \left(\nabla \log \pi(y^+|x) - \nabla \log \pi(y^-|x) \right) \right]$$

Pairs the model currently ranks the wrong way round get the largest weight.

CAVEATS

Offline. It learns from a fixed dataset; online or iterative DPO samples fresh pairs from the current policy.

Margins, not likelihoods. The margin can grow while the chosen response's probability also falls.

Overfitting. Deterministic preferences push log-ratios toward ±∞; IPO adds a fixed target margin.

IPO: Regress the Margin, Don’t Maximize It

Same data and models as DPO; a squared loss stops the margin at a finite target.

$$h_\pi = \log \pi(y^+|x)/\pi_{ref}(y^+|x) - \log \pi(y^-|x)/\pi_{ref}(y^-|x)$$

$$\mathbf{L_{IPO} = \mathbb{E}\left[\left(h_\pi - 1/(2\tau) \right)^2 \right]}$$

DPO, for comparison: $L_{DPO} = -\mathbb{E}[\log \sigma(\beta \cdot h_\pi)]$

```
def ipo_loss(pi_pos, pi_neg, ref_pos, ref_neg, tau=0.1):
    h = (pi_pos - ref_pos) - (pi_neg - ref_neg)    # margin
    return ((h - 1 / (2 * tau)) ** 2).mean()

# DPO's loss keeps falling as h grows: it rewards h -> inf
# IPO's gradient flips sign once h overshoots 1/(2*tau)
```

AT A GLANCE

Diagnosis. RLHF and DPO use Ψ = logit, unbounded at $p = 1$: if each pair is seen once, $\hat{p} = 1$ and the optimum sets $\pi(y^-) = 0$ for any β.

Fix. Ψ = identity is bounded, so the KL term keeps working: the optimal margin is $(\hat{p} - \tfrac{1}{2})/\tau$.

Cost. Same as DPO: pairs, the policy and a frozen reference; τ plays β’s role. The paper itself tests only toy bandits.

Think. If 80% of annotators prefer y^+, show that DPO’s optimal margin is $\text{logit}(0.8)/\beta$ and IPO’s is $0.3/\tau$. Now let $\hat{p} \to 1$.

KTO: Align with Thumbs Up and Thumbs Down

No pairs: each output is labeled desirable or undesirable and valued against a reference point.

$r_\theta = \log \pi(y|x)/\pi_{ref}(y|x)$, z_0 = batch KL estimate, no gradient

$v(x, y) = \lambda_D \cdot \sigma(\beta\,(r_\theta - z_0))$ if y is desirable

$v(x, y) = \lambda_U \cdot \sigma(\beta\,(z_0 - r_\theta))$ if y is undesirable

$L_{KTO} = \mathbb{E}[\,\lambda_y - v(x, y)\,]$

```
def kto_loss(pi, ref, good, z0, beta=0.1, lam_d=1.0, lam_u=1.0):
    r = pi - ref   # implied reward of each output
    vg = lam_d * torch.sigmoid(beta * (r - z0))   # desirable
    vb = lam_u * torch.sigmoid(beta * (z0 - r))   # undesirable
    return torch.where(good, lam_d - vg, lam_u - vb).mean()
# z0 = max(0, mean(pi - ref)) over mismatched pairs, detached
```

AT A GLANCE

Data. One thumbs-up or thumbs-down label per output, no pairs. Imbalance is fine: keep $\lambda_D n_D / (\lambda_U n_U)$ in $[1, 4/3]$.

Prospect theory. Value is relative to a reference point z_0, with separate weights for gains and losses; σ keeps the loss stable.

Reported. Matches or beats DPO from 1B to 30B, and matches it with up to 90% fewer desirable examples.

Think. σ saturates. Which examples get almost no gradient, and why might that filter out noisy labels?

SimPO: Average Log-Probability as Reward

No reference model: score responses the way decoding ranks them, and demand a margin.

$$r(x, y) = (\beta / |y|) \cdot \log \pi(y \mid x)$$

β times the average token log-probability: Block II's confidence score

$$\mathbf{L_SimPO = -E[\ \log \sigma(\ r(x, y^{+}) - r(x, y^{-}) - \gamma\ )\ ]}$$

```
def simpo_loss(pi_pos, pi_neg, len_pos, len_neg,
               beta=2.0, gamma=1.0):
    r_pos = beta * pi_pos / len_pos   # mean token log-prob
    r_neg = beta * pi_neg / len_neg
    return -F.logsigmoid(r_pos - r_neg - gamma).mean()
# no ref_* inputs: one model, no reference forward pass
```

AT A GLANCE

Mismatch. DPO's reward, β log π/π_ref, is not what decoding ranks by: after DPO the two agree on only about half of training pairs.

Ingredients. Length normalization (without it, outputs grow long and repetitive) and a target margin γ; β ≈ 2–2.5, γ ≈ 0.3–1.6.

Reported. Up to +6.4 points over DPO on AlpacaEval 2 and +7.5 on Arena-Hard. No KL anchor; the paper notes drops on math (GSM8K).

Think. Why does SimPO need β ≈ 2 when DPO uses β ≈ 0.1? Compare a per-token average with a summed log-ratio.

ORPO: Preference Optimization Inside SFT

One stage, one model: the SFT loss plus a small odds-ratio penalty on the rejected answer.

$$\text{odds}(y \mid x) = P / (1 - P), \quad P = \exp(\text{mean token log-prob of } y)$$
$$L_{OR} = -\log \sigma(\log \text{odds}(y^+ \mid x) - \log \text{odds}(y^- \mid x))$$
$$\mathbf{L_{ORPO} = E[\, L_{SFT}(y^+) + \lambda \cdot L_{OR} \,]}$$

```
def orpo_loss(nll_pos, lp_pos, lp_neg, lam=0.1):
    # lp_*: mean token log-prob; nll_pos: SFT loss on y+
    log_odds = (lp_pos - torch.log1p(-lp_pos.exp())) \
             - (lp_neg - torch.log1p(-lp_neg.exp()))
    return (nll_pos - lam * F.logsigmoid(log_odds)).mean()
```

AT A GLANCE

Problem. SFT on chosen answers also lifts rejected ones: in the paper both log-probabilities rise together.

Mechanism. The rejected answer's gradient carries $1/(1 - P)$, so the penalty bites hardest when it is likely; λ is small (TRL: 0.1).

Reported. Mistral-ORPO-β (7B, UltraFeedback only): 12.20% AlpacaEval 2.0, 66.19% IFEval, 7.32 MT-Bench.

Think. With no reference model and no KL term, what keeps ORPO from drifting? And what anchors SimPO?

PREFERENCE OPTIMIZATION · 2022–2024

A Family of Preference Objectives

Online RL versus direct alignment: what each needs at training time.

Method	Needs at training time	Key idea
PPO-RLHF	RM, value model, reference; online samples	clipped policy gradient on the RM reward, with a KL penalty
RLOO, GRPO	reward, reference; online samples	baseline from other samples of the same prompt; no critic
DPO	preference pairs, reference	logistic loss on the policy-vs-reference log-ratio margin
IPO	preference pairs, reference	squared loss toward a fixed margin; resists overfitting
KTO	unpaired good / bad labels, reference	prospect-theory utility per example; no pairs needed
SimPO	preference pairs only	length-normalized log-prob as reward, plus a target margin
ORPO	preference pairs only	odds-ratio penalty added to the SFT loss; one stage

Ouyang 2022 · Ahmadian 2024 · Shao 2024 · Rafailov 2023 · Azar 2023 · Ethayarajh 2024 · Meng 2024 · Hong 2024

EXTENSION · CURRENT PRACTICE

RLHF in Modern Post-Training

Preferences shape style and safety; verifiable rewards now drive reasoning.

Model	Post-training stages
InstructGPT (2022)	SFT → reward model → PPO
Llama 2-Chat (2023)	SFT → separate helpfulness and safety reward models → rejection sampling, then PPO
Llama 3 (2024)	several rounds of SFT, rejection sampling and DPO
Tülu 3 (2024), Olmo 3 (2025)	SFT → DPO → RL with verifiable rewards, with open data and code
DeepSeek-R1 (2025)	cold-start SFT → reasoning RL (GRPO, rule-based rewards) → rejection-sampling SFT → RL with rule-based and preference rewards

AI feedback. Constitutional AI (2022): a model guided by written principles labels harmlessness.

Rubrics as rewards. Rubric-scored LLM judges extend RL beyond verifiable domains (2025).

CHAPTER 6.1 · BRIDGE TO NEXT SESSION

RLHF and RLVR Side by Side

The same loop — generate, score, update — with a different source of reward.

	RLHF · today	RLVR · Chapter 6
Reward source	learned reward model trained on human preferences	deterministic verifier: correct = 1, incorrect = 0
Domains	open-ended: helpfulness, style, harmlessness	checkable: math, code
Extra models	a reward model, often as large as the policy	none beyond the verifier
Labels	human or AI comparisons; noisy (73–77% agreement)	free and reproducible, given reference answers
Main risk	over-optimizing an imperfect proxy	verifier loopholes (Session 5: false positives)
Typical optimizer	PPO online; DPO offline	GRPO (Chapter 6) and PPO variants

Next session: Chapter 6 keeps today's loop and swaps the reward model for the Chapter 3 verifier.

THINKING PROBLEMS · BLOCK III · 8 MIN

Thinking Problems

Work in pairs; we will discuss two of the three.

Q1 · INVARIANCE

Show that the Bradley–Terry likelihood is unchanged by $r(x, y) \to r(x, y) + c(x)$. Why does this matter once r is used as an RL reward across prompts, and how does a baseline deal with it?

Q2 · THE OPTIMAL POLICY

Derive $\pi^*(y \mid x) \propto \pi_{\text{ref}}(y \mid x) \cdot \exp(r(x, y) / \beta)$ from the KL-regularized objective. Hint: rewrite it as $-\beta \cdot \mathrm{KL}(\pi \,\|\, \pi^*)$ plus a constant. What happens as $\beta \to 0$ and $\beta \to \infty$?

Q3 · GOODHART IN PRACTICE

Thumbs-up feedback is abundant and free. Give two reasons it can be a worse reward than paid pairwise comparisons, and propose a safeguard for each.

Block III in Four Lines

1. RLHF learns a reward from human comparisons (Bradley–Terry), then optimizes the policy against it on a KL leash to the SFT model.

2. PPO keeps updates stable with clipped ratios and a learned value baseline, at the cost of four LLM-sized models.

3. The reward is a proxy: over-optimization shows up as length bias, sycophancy and lost diversity.

4. DPO solves the same objective with a classification loss; RLVR, next, replaces the learned reward with a verifier.

READING AND PRACTICE

Before Next Session

Chapter 6 turns today's two threads — log-probabilities and rewards — into GRPO.

READ

Chapter 6, "Training reasoning models with reinforcement learning": RLVR with GRPO.

RUN

Book exercises 4.3–4.4 (tiebreaks, early stopping) and 5.3–5.4 (logprob tiebreak, best-of-N) on a MATH-500 subset.

OPTIONAL PAPERS

Ouyang et al. (2022), InstructGPT; Rafailov et al. (2023), DPO; Gao et al. (2023), reward over-optimization.

Inference-Time Scaling

Raschka, S. *Build a Reasoning Model (From Scratch)*. Manning. Chapters 4–6, Appendix A.

Kojima et al. (2022). Large Language Models are Zero-Shot Reasoners. NeurIPS.

Wang, X. et al. (2023). Self-Consistency Improves Chain of Thought Reasoning in Language Models. ICLR.

Holtzman et al. (2020). The Curious Case of Neural Text Degeneration. ICLR.

Nguyen et al. (2025). Turning Up the Heat: Min-p Sampling for Creative and Coherent LLM Outputs. ICLR.

Feng et al. (2023). Towards Revealing the Mystery behind Chain of Thought: A Theoretical Perspective. NeurIPS.

Merrill & Sabharwal (2024). The Expressive Power of Transformers with Chain of Thought. ICLR.

Snell et al. (2024). Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters. arXiv:2408.03314.

Brown et al. (2024). Large Language Monkeys: Scaling Inference Compute with Repeated Sampling. arXiv:2407.21787.

Kadavath et al. (2022). Language Models (Mostly) Know What They Know. arXiv:2207.05221.

Madaan et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS.

Huang et al. (2024). Large Language Models Cannot Self-Correct Reasoning Yet. ICLR.

Shao et al. (2025). DeepSeekMath-V2: Towards Self-Verifiable Mathematical Reasoning. arXiv:2511.22570.

Fu et al. (2025). Deep Think with Confidence. arXiv:2508.15260.

Wang, J. et al. (2025). Think Deep, Think Fast: Investigating Efficiency of Verifier-free Inference-time-scaling Methods. arXiv:2504.14047.

REFERENCES · BLOCK III

RLHF and Preference Optimization

Christiano et al. (2017). Deep Reinforcement Learning from Human Preferences. NeurIPS.

Ziegler et al. (2019). Fine-Tuning Language Models from Human Preferences. arXiv:1909.08593.

Stiennon et al. (2020). Learning to Summarize from Human Feedback. NeurIPS.

Ouyang et al. (2022). Training Language Models to Follow Instructions with Human Feedback. NeurIPS.

Bai et al. (2022). Constitutional AI: Harmlessness from AI Feedback. arXiv:2212.08073.

Schulman et al. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.

Rafailov et al. (2023). Direct Preference Optimization. NeurIPS.

Gao, Schulman & Hilton (2023). Scaling Laws for Reward Model Overoptimization. ICML.

Sharma et al. (2024). Towards Understanding Sycophancy in Language Models. ICLR.

Singhal et al. (2024). A Long Way to Go: Investigating Length Correlations in RLHF. COLM.

Kirk et al. (2024). Understanding the Effects of RLHF on LLM Generalisation and Diversity. ICLR.

Röttger et al. (2024). XSTest. NAACL. OpenAI (2025). Expanding on What We Missed with Sycophancy.

IPO: Azar et al. (2023). KTO: Ethayarajh et al. (2024). SimPO: Meng et al. (2024). ORPO: Hong et al. (2024). RLOO: Ahmadian et al. (2024).

Shao et al. (2024). DeepSeekMath (GRPO). arXiv:2402.03300.

DeepSeek-AI (2025). DeepSeek-R1. arXiv:2501.12948.

Touvron et al. (2023). Llama 2. Llama Team (2024). The Llama 3 Herd of Models.

Lambert et al. (2024). Tülu 3. Olmo Team (2025). Olmo 3.

Gunjal et al. (2025). Rubrics as Rewards. arXiv:2507.17746; ICLR 2026.

Lambert, N. Reinforcement Learning from Human Feedback. rlhfbook.com.